Práctica de Laboratorio 4: Gestión de Interrupciones y Excepciones

Richard Vasquez

13 de octubre de 2026

1. Respuestas al Cuestionario

1.1. 1. Ciclo de Interrupción de Hardware

(El diagrama lógico conceptual es el siguiente): 1) **Evento:** El usuario presiona una tecla. El hardware levanta una línea de IRQ (Interrupt Request). 2) **CPU Acepta:** El CPU termina de ejecutar la instrucción actual en el pipeline. 3) **Respaldo PC:** Se guarda la dirección de la siguiente instrucción en el registro `EPC` (Coprocesador 0, reg $14). 4) **Salto al Kernel:** El PC se fuerza a la dirección `0x80000180` (Vector de Excepción). 5) **Rutina (ISR):** Se ejecuta el código para leer la tecla y guardarla. 6) **Retorno:** La instrucción `eret` restaura el PC desde el `EPC`, volviendo al programa original invisiblemente.

1.2. 2. Sondeo (Polling) vs. Interrupciones

El sondeo obliga al procesador a preguntar repetidamente al dispositivo si necesita atención (espera activa, desperdicia energía y CPU). Las interrupciones permiten al procesador ejecutar otras tareas y es el dispositivo quien envía una señal eléctrica (IRQ) a la CPU solo cuando hay datos listos o sucede un evento.

1.3. 3. Ventajas de las Interrupciones

Minimizan el uso de CPU en tareas de espera, reducen drásticamente el consumo de energía y permiten la creación de Sistemas Operativos multitarea, donde la CPU puede atender múltiples procesos y periféricos aparentemente al mismo tiempo.

1.4. 4. Registros especiales en MIPS32 (Coprocesador 0)

- `Status ($12)`: Controles de máscara para habilitar o deshabilitar interrupciones globales.
- `Cause ($13)`: Indica el código del motivo por el cual ocurrió la excepción o interrupción.
- `EPC ($14)`: (Exception Program Counter) Guarda la dirección a la cual retornar al terminar.

1.5. 5. Necesidad de guardar el contexto

La Rutina de Servicio (ISR) utiliza registros de uso general para trabajar (ej. `$t0, $t1`). Si no respaldamos los valores que estos registros tenían antes de la interrupción, corromperemos los datos del programa principal, causando un comportamiento impredecible al ejecutar `eret`.

1.6. 6. Excepciones en MIPS32

a) Situaciones: 1. Desbordamiento aritmético en suma (`add`). 2. Llamada al sistema (`syscall`). 3. Fallo de dirección (intentar acceder a memoria no alineada o prohibida). 4. Opcode inválido (instrucción desconocida).
b) Etapas del Pipeline:

- **IF (Búsqueda):** Falla si el PC intenta leer de una dirección no permitida (Instruction Address Exception).
- **ID (Decodificación):** Falla si la instrucción no existe.
- **EX (Ejecución):** Falla por desbordamiento aritmético en la ALU.
- **MEM (Memoria):** Falla en un `lw/sw` si la dirección de datos no está alineada.

1.7. 7. Estrategias de tratamiento

a) Diferencias: Las **Interrupciones** son asíncronas porque provienen de hardware externo y pueden ocurrir en cualquier ciclo de reloj. Las **Excepciones** son síncronas; son causadas directamente por la instrucción que se está ejecutando en ese momento.
b) Ejecución y EPC: En MIPS32 clásico, todas las excepciones redirigen a un único manejador (`0x80000180`). Allí, el software lee el registro `Cause` para derivar la ejecución (Estrategia centralizada). El registro `EPC` sirve para saber dónde retomar el hilo principal (usando `eret`).

1.8. 8. Habilitación de interrupciones

a) Habilitación:

- **Teclado:** Escribiendo `1` en el bit 1 del registro `0xFFFF0000` (Receiver Control).
- **Procesador:** Configurando en `1` el Bit 0 (Interrupt Enable) del registro `Status` ($12 del Coproc 0).

b) Procesador aislado: Si se habilitan en el dispositivo pero no en el registro `Status` de la CPU, las interrupciones quedan .enmascaradas". El periférico enviará la señal, pero el procesador simplemente la ignorará.

1.9. 9. Procesamiento de interrupciones de Reloj

a) Paso a paso: El temporizador llega a cero (Hardware) → Envía IRQ al procesador → CPU pasa control a `0x80000180` → ISR del sistema operativo (Software) incrementa contador de "Ticks"→ ISR usa `eret` para volver. Se guardan los registros temporales y

el PC.
b) Importancia: Proteger la transparencia. El programa interrumpido nunca debe darse cuenta de que el procesador se detuvo para atender el reloj.

1.10. 10. Interrupciones de reloj y control de ejecución

a) Uso del reloj: Permite implementar el "Time Slicing"(Planificación). El SO configura el reloj cada X milisegundos. Si un programa está en un bucle infinito, el reloj interrumpe a la fuerza y el SO le quita el control del CPU para dárselo a otro proceso.
b) Terminación temprana: Si el programa finaliza antes, ejecuta un `syscall` de salida (ej. `exit`). El SO retoma el control, cancela el temporizador de ese programa y reprograma el reloj para el siguiente proceso.

1.11. 11. Análisis y Discusión de los Resultados

Se demostró en MARS la superioridad de la asincronía. Mientras el bucle principal (‘main‘) ejecutaba un chequeo del tiempo (‘syscall 30‘), el programa era capaz de registrar pulsaciones de teclado en segundo plano sin interrumpir su conteo de 20 segundos. El uso de la directiva ‘.ktext‘ permitió aislar la lógica de recolección de teclado (buffer) protegiendo el flujo principal de ejecución.